

Legibilidad del código

⚠ REGLAS

MAXIMO 120 CARACTERES POR LINEA, 4 ESPACIOS DE IDENTACION (TAB)
SEGUN JERARQUIA

Variables

Inicialización y datos

LAS VARIABLES SE INICIALIZAN EN EL MENOR ALCANCE POSIBLE, ES DECIR, DONDE SE VAN A USAR (SI SE USAN EN TODO EL PROGRAMA SE INICIALIZAN AL PRINCIPIO)

C++ **C++** *Inicialización de variables*

```
1  const char INI_CHAR = 0;  
2  const int INI_INT = 0;  
3  const bool INI_BOOL = 0;  
4  const float INI_FLOAT = 0f;  
5  const double INI_DOUBLE = 0.0;  
6  const long INI_LONG = 0L;  
7  const short INI_SHORT = 0;  
8  
9  //STRINGS  
10 const std::string INI_STRING = "a";
```

Datos Primitivos C++

Son datos muy similares a los [Datos Primitivos java](#) pero pueden tener modificadores, y tienen tamaños dependientes del compiler o la plataforma

```

C++  C++  Datos primitivos C++
1   char dato 1 = 0;
2
3   short dato2 = 0;
4
5   int dato 3 = 0;
6
7   long dato 4 = 0L;
8
9   long long dato_5 = 0LL;
10
11  float dato6 = 0f;
12
13  double dato7 = 0.0;
14
15  bool dato8 = false;
16
17  //STRINGS
18
19  std::string dato9 = "a";

```

```

C++  C++  Modificadores de datos
1   signed(default) int x = -1; //Significa que el rango del int es
   completo (positivos y negativos)
2   unsigned int x = 1; //Solo puede ser mayor/igual a 0
3   short int x = 0; //El dato ocupa max 16bits
4   long int x = 0; //El dato ocupa entre 32 y 64 bits
5   long long int x = //El dato ocupa 64 bits

```

Nomenclatura de variables, métodos y clases

camelCase variables y métodos

PascalCase clases

NOMBRES DESCRIPTIVOS, NO INTERPRETABLES

C++

C++

Nomenclatura

```

1  class SaludarProgramador //PascalCase
2      void saludoProgramador(){ //camelCase
3          std::string nombreProgramador = "Ignacio"; //camelCase
4          std::cout << "Hola," << nombreProgramador; // Printea Hola
5      ignacio
6      }

```

Comentarios

Seguimos el estándar Doxygen

Estructura:

C++

C++

Doxygen

```

1  /**
2      * @brief Resumen de la clase /metodo
3      * Comentarios genericos sobre la clase/metodo
4      * @param [nombre] parametros de la estructura
5      * @return[nombre] valor devuelto
6      */

```

Ejemplo:

C++

C++

Comentarios Doxygen

```

1  /**
2      * @brief Clase que se usa para saludar al programador
3      * @param nombreProgramador String que almacena el nombre del programador
4      */
5
6  class SaludarProgramador //PascalCase
7      void saludoProgramador(){ //camelCase
8          std::string nombreProgramador = "Ignacio"; //camelCase
9          std::cout << "Hola," << nombreProgramador; // Printea Hola
10     ignacio
11     }

```

Manejo de errores

LAS INSTRUCCIONES CRITICAS SE PROTEGEN CON try/catch

⚠ INSTRUCCIONES A PROTEGER

Operaciones de entrada/salida

Sockets de transmision (procesos)

Configuraciones de sensores/Operaciones de carga de archivos

Operaciones con memoria (Lilypad tiene muy poca memoria)

BLOQUE MAIN

Gestión de Memoria

 **PROHIBIDO**

NO SE PUEDE USAR NEW/DELETE ;NUNCA!